

Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 16, 2025
Milestone 4

Files

<https://github.com/CGalloway3/CST-250-Projects/tree/master/Milestone4>

Video Part 1

<https://www.loom.com/share/320fe85e85534d029625da4563f3b1a9>

Video Part 2

<https://www.loom.com/share/f833a8f1d59d408597bfc65ae60550a8>

FLOWCHART

/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/16/2025
* Mine Sweeper GUI App
* Milestone 4
* References:
*/

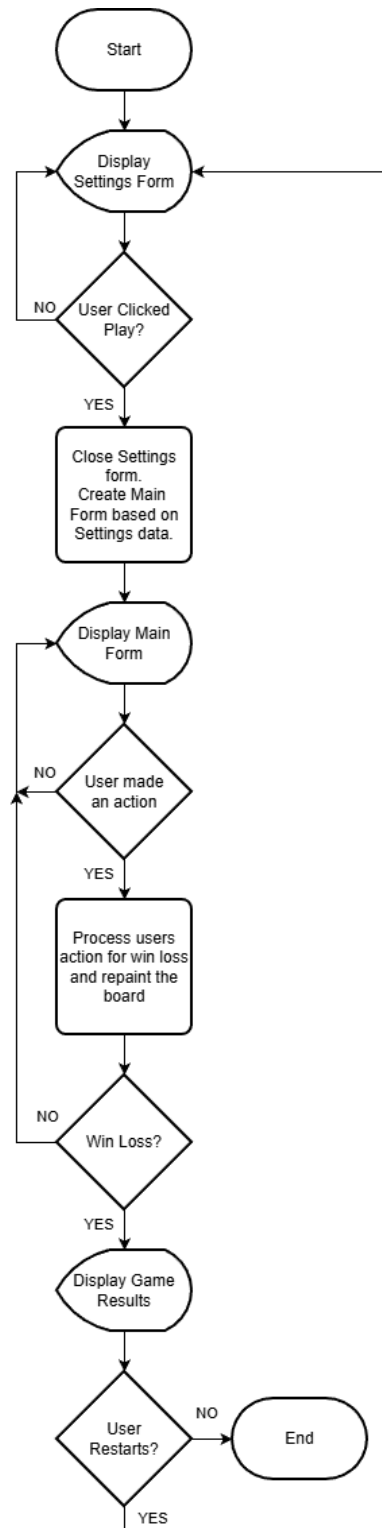


Figure 1: Flowchart of Logic in Milestone 4 Mine Sweeper GUI App

UML Class Diagram

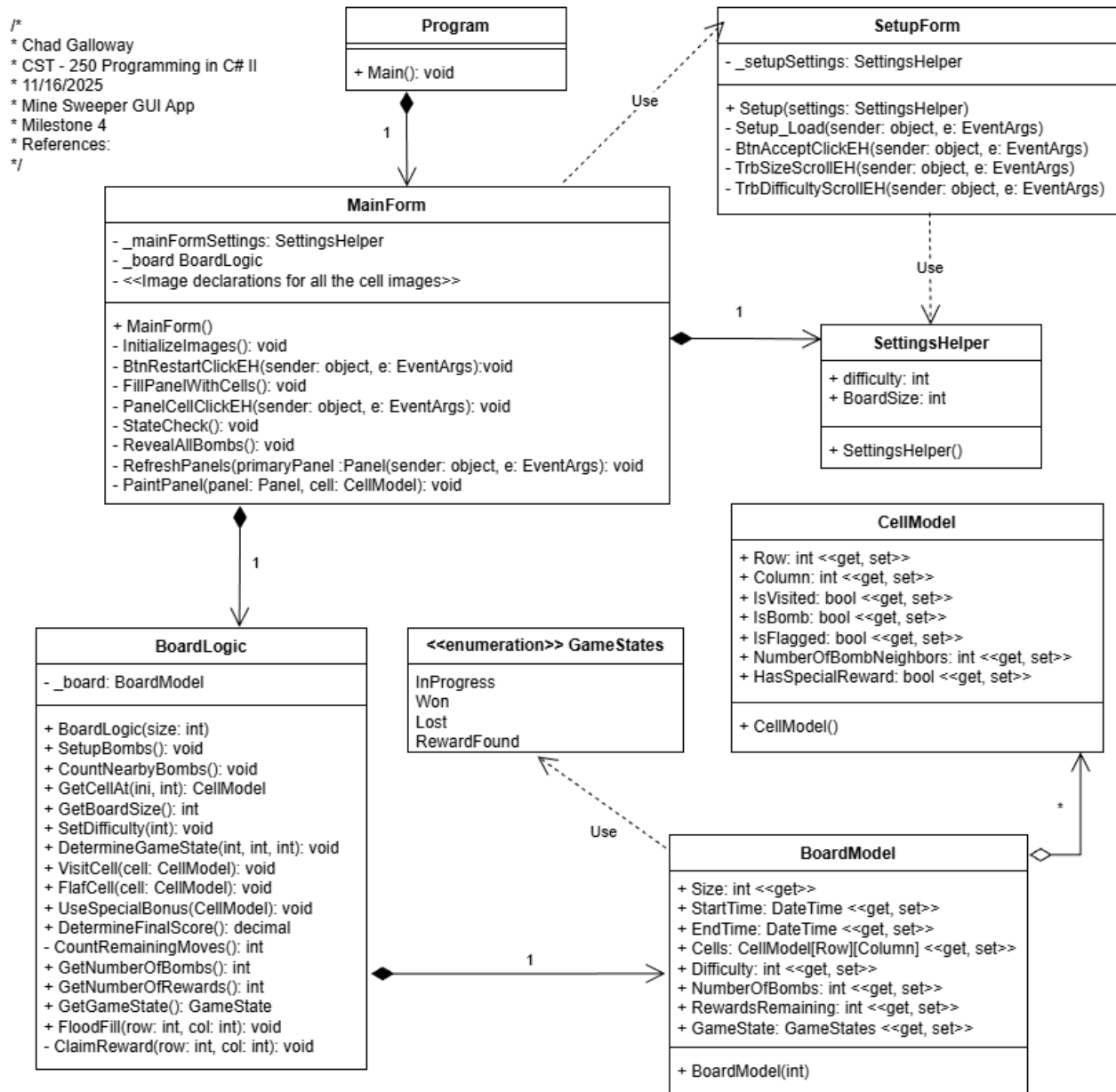


Figure 2: UML Class Diagram for Milestone 4 Mine Sweeper GUI App

Screen Shots

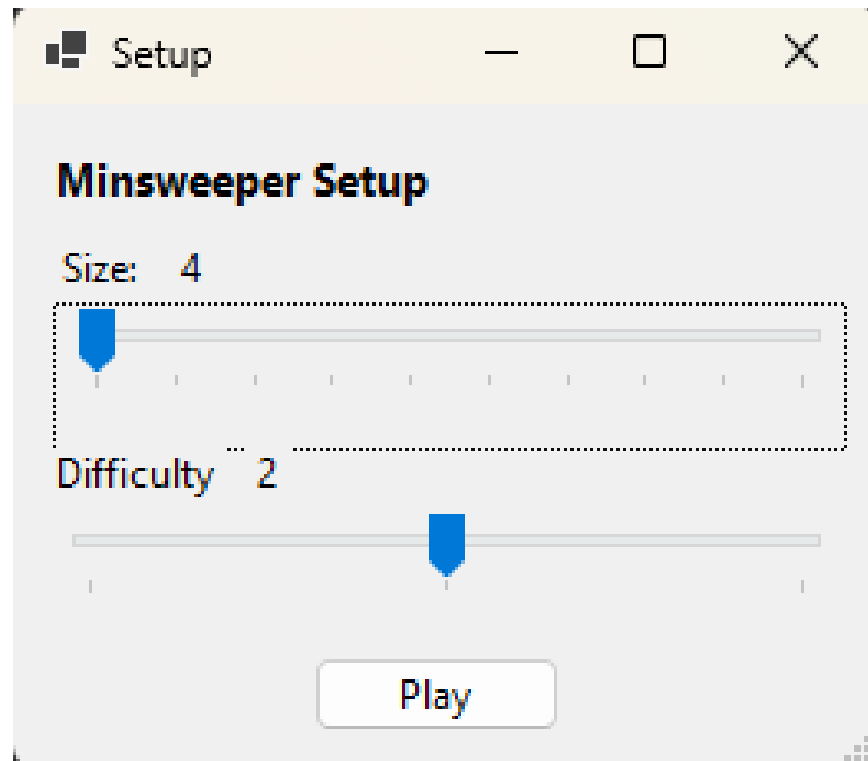


Figure 3: Screenshot of the setup window that loads when the App starts.

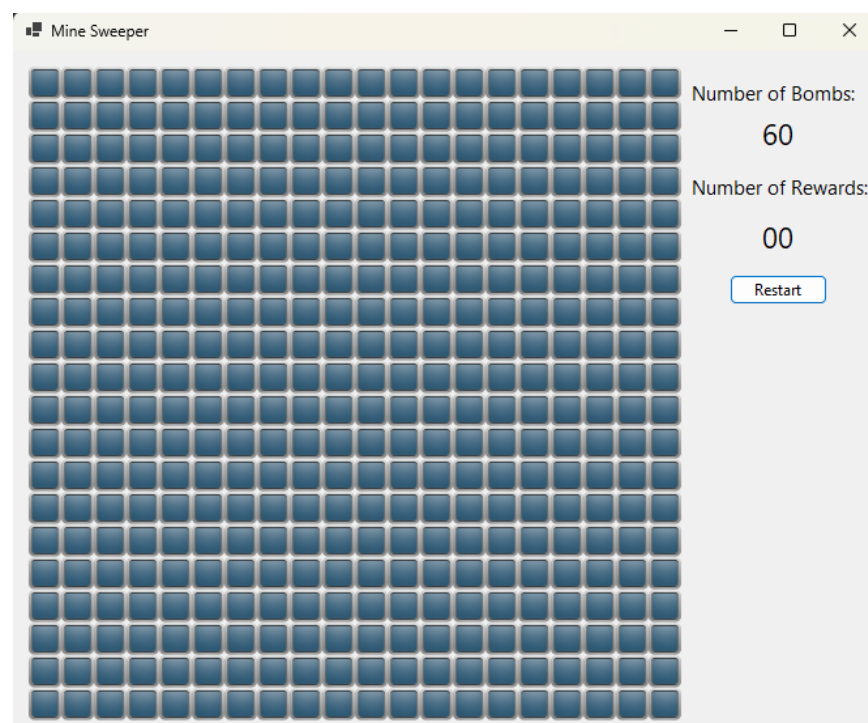


Figure 4: Screenshot of main form with size 20 and difficulty 2

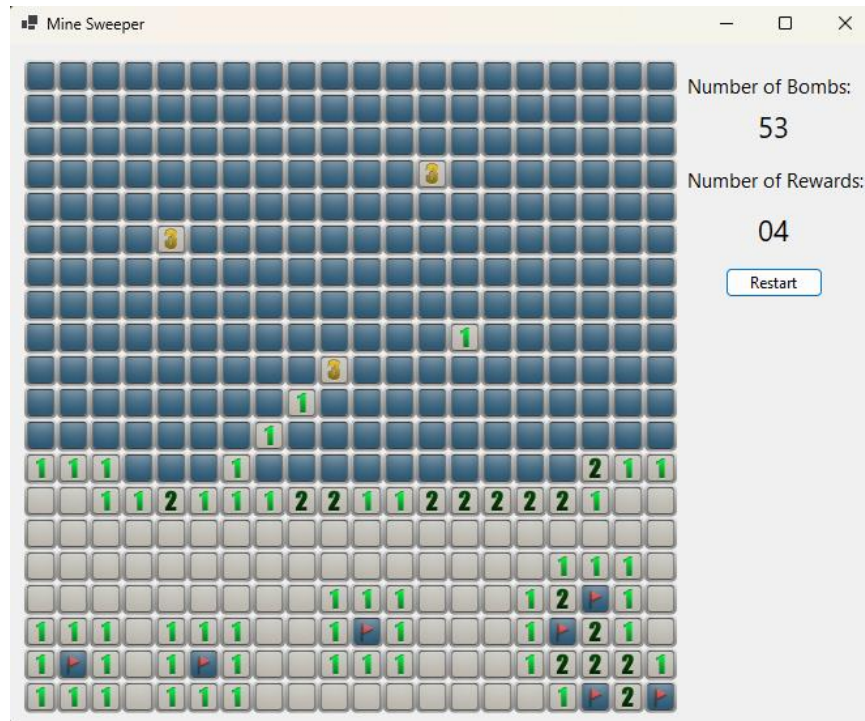


Figure 5: Screenshot of an example of the flood fill in action. The user reveals a cell by left clicking.

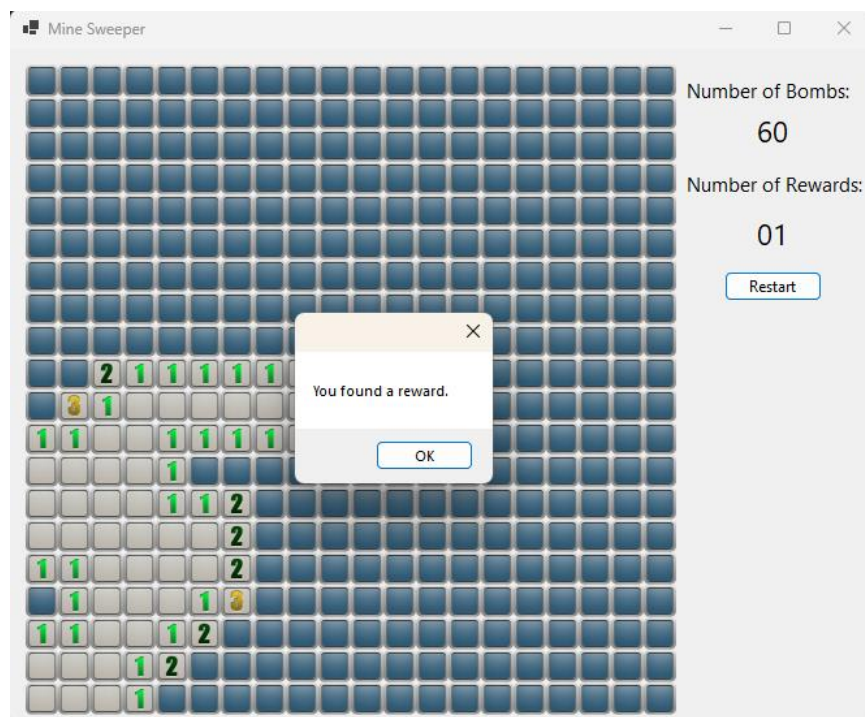


Figure 6: Screenshot of the notification the user receives when they find a reward.

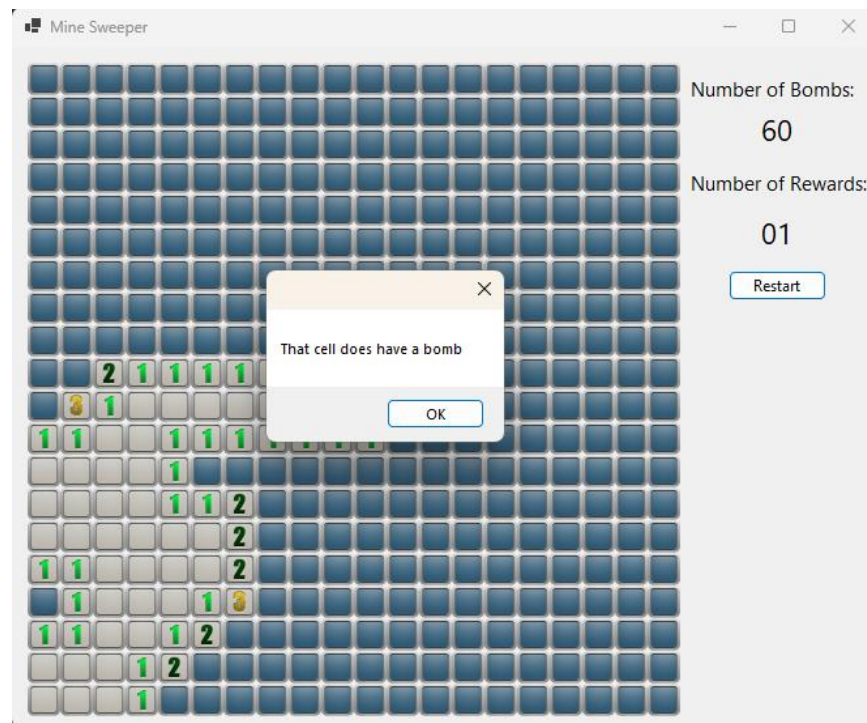


Figure 7: Screenshot of the notification the user receives when a reward is used. The user can use a reward by middle mouse button clicks (or any other button besides left or right, the use reward functionality is a fall through of all remaining click args that are not left or right.)



Figure 8: Screenshot of a highlight placed on cells that have had rewards used on them. In my testing I found it very difficult to know the exact cell you clicked to use a reward. After acknowledging the MessageBox with the result of the rewards usage it was often difficult to know where you had used the reward. I added this highlight feature in so it was easier to know where you used the reward. Any subsequent clicks on that cell removes the highlight.

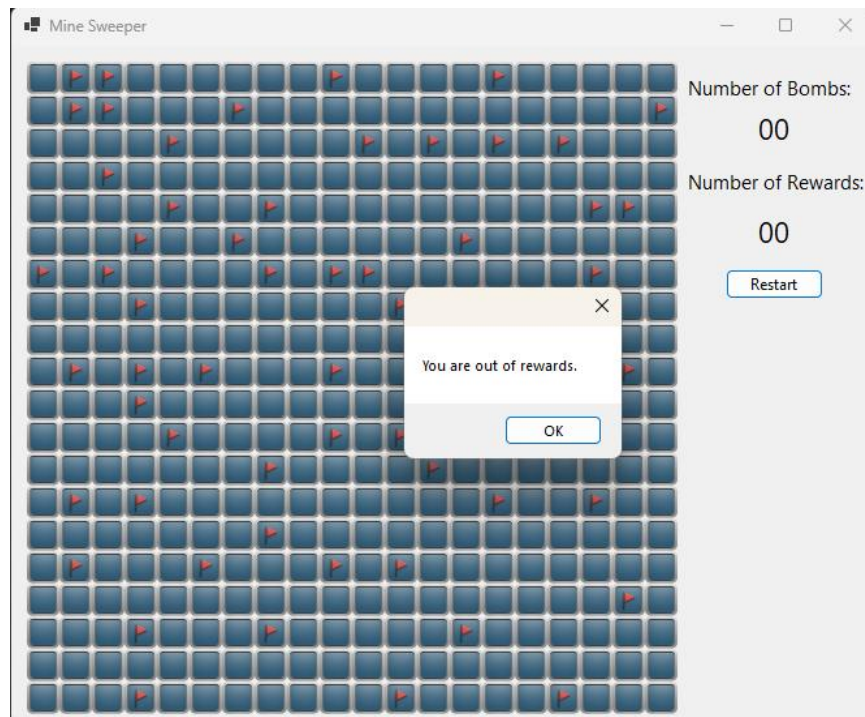


Figure 9: Screenshot of catching the cases where a user has no rewards but still attempts to use one.



Figure 10: Screenshot of close up of the cells with bomb flags marked on them. The user can place and remove bombs by right clicking. The user is not allowed to place more flags than there are bombs and they cannot place a flag in an already revealed cell. These two cases are displayed in the next two Figures 11 and 12.

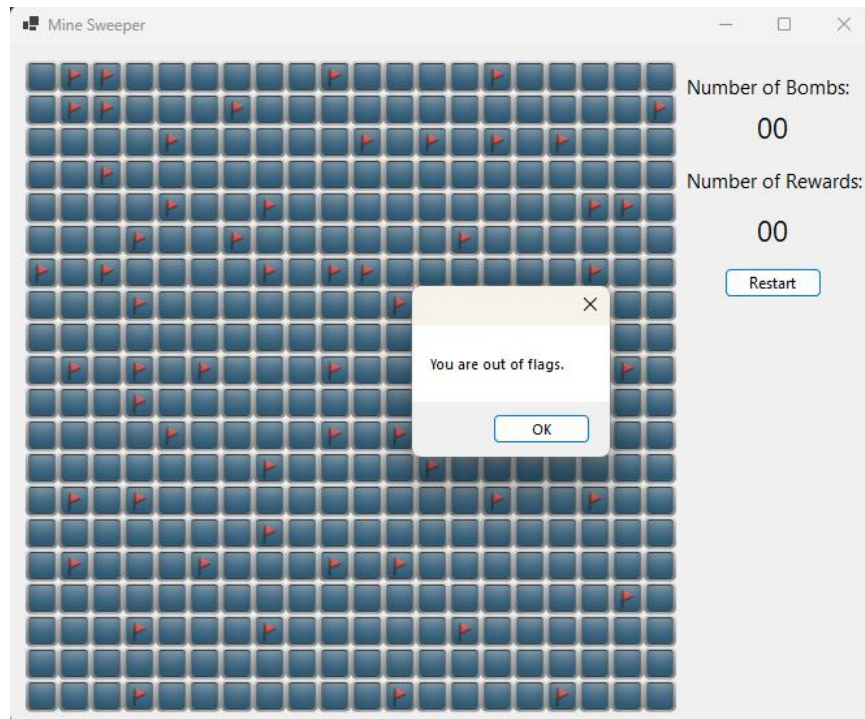


Figure 11: Screenshot of message the user gets if they have no remaining flags to use.

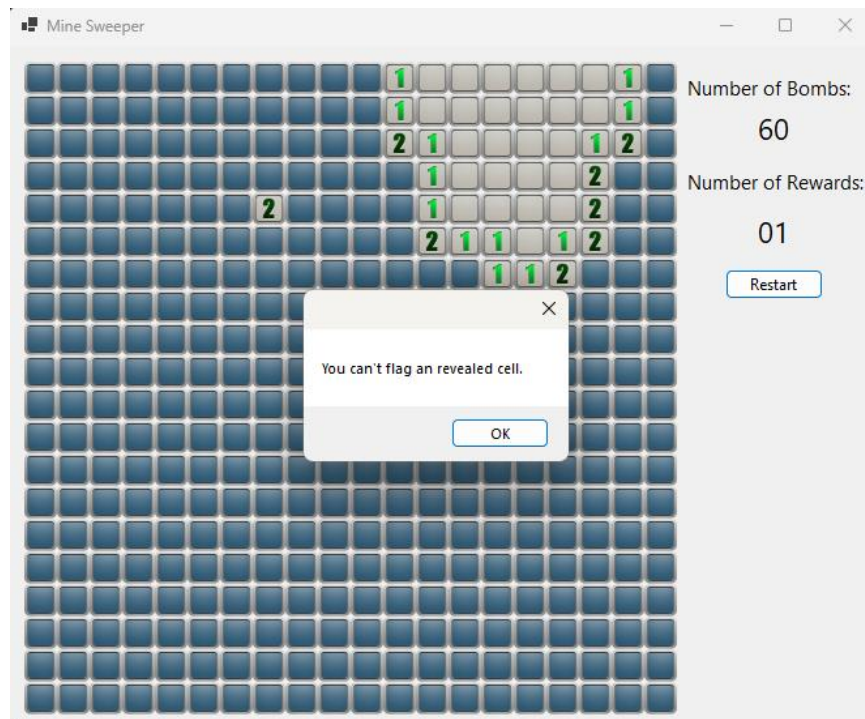


Figure 12: Screenshot of message displayed to the user when they try to flag an already revealed cell.

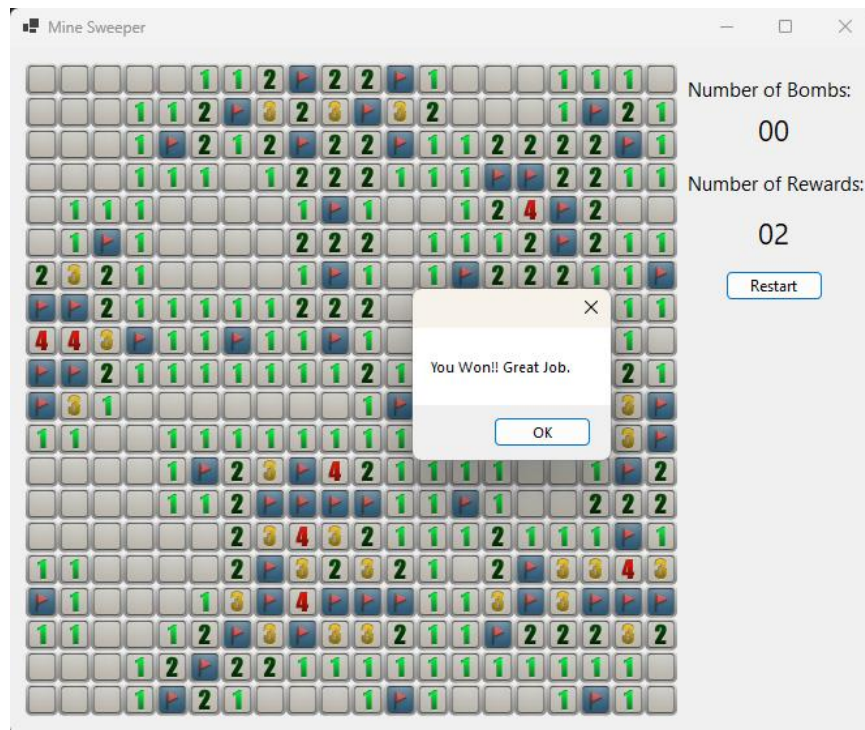


Figure 13: Screenshot of a Win condition

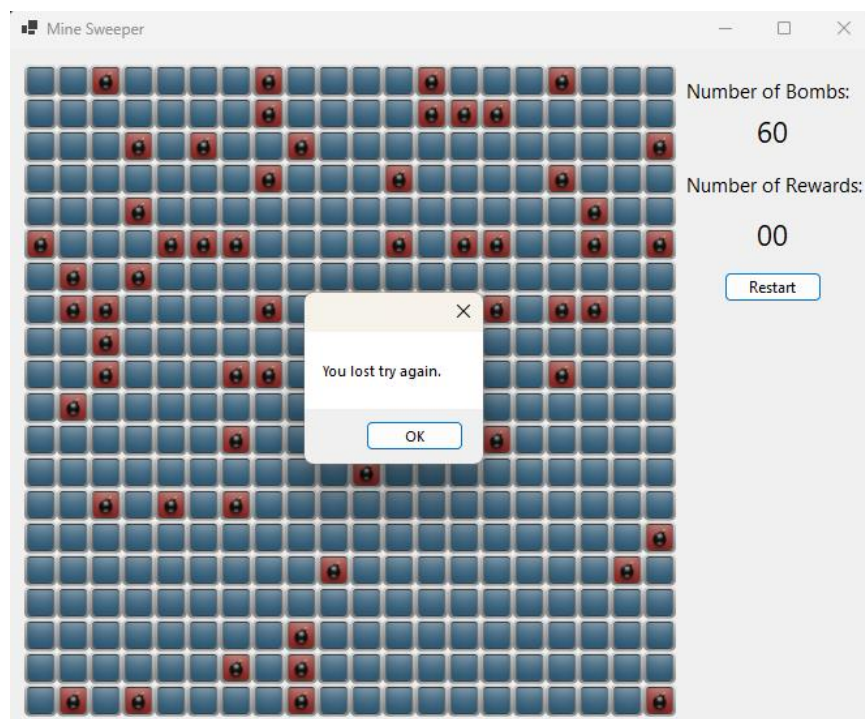


Figure 14: Screenshot of a loss condition.

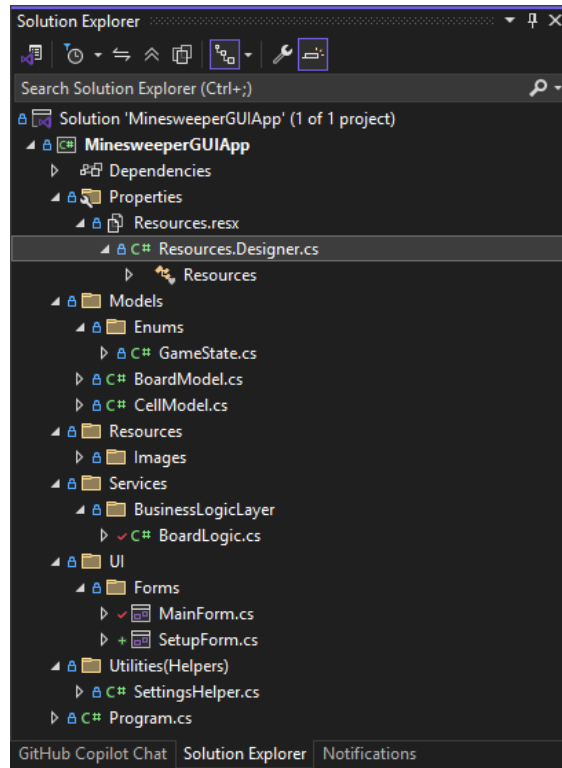


Figure 15: Screenshot of the Solution Explorer layout for the project.

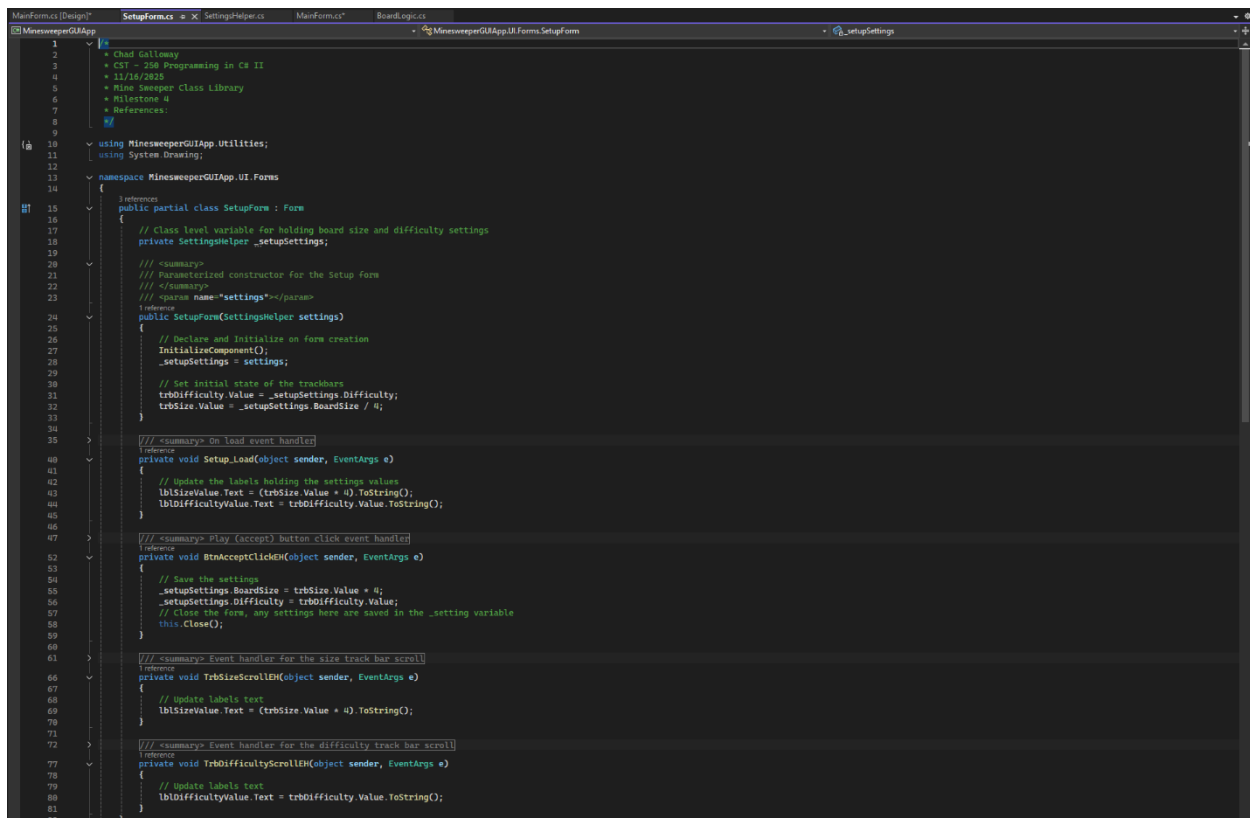


Figure 16: Screenshot of the code for the setup form.

```

1  //
2  //
3  //
4  //
5  //
6  //
7  //
8  //
9  //
10 namespace MinesweeperGUIApp.Utilities
11 {
12     /// <summary>
13     /// Class to hold the values of the settings page
14     /// </summary>
15     /// </summary>
16     public class SettingsHelper
17     {
18         // Properties
19         public int Difficulty { get; set; }
20         public int BoardSize { get; set; }
21
22         /// <summary>
23         /// Default constructor
24         /// </summary>
25         public SettingsHelper()
26         {
27             Difficulty = 1;
28             BoardSize = 4;
29         }
30     }
31 }

```

Figure 17: Screenshot of the code for the settings helper class. This class makes it easy to pass data between the two forms.

```

1  //
2  //
3  //
4  //
5  //
6  //
7  //
8  //
9  //
10 using MinesweeperGUIApp.Models;
11 using MinesweeperGUIApp.Services.BusinessLogicLayer;
12 using MinesweeperGUIApp.Services.BusinessLogicLayer;
13 using MinesweeperGUIApp.Utilities;
14 using System.Windows.Forms;
15
16 namespace MinesweeperGUIApp.UI.Forms
17 {
18     public partial class MainForm : Form
19     {
20         // Declare settings and board objects
21         private SettingsHelper _mainFormSettings;
22         private BoardLogic _board;
23
24         // Declare the variables for image resources used to paint cells
25         private Image _blankCellImage;
26         private Image _oneNeighborImage;
27         private Image _twoNeighborsImage;
28         private Image _threeNeighborsImage;
29         private Image _fourNeighborsImage;
30         private Image _fiveNeighborsImage;
31         private Image _sixNeighborsImage;
32         private Image _sevenNeighborsImage;
33         private Image _eightNeighborsImage;
34         private Image _nineNeighborsIsBombImage;
35         private Image _flaggedCellImage;
36         private Image _hiddenCellImage;
37
38         /// <summary> Entry Point
39         /// </summary>
40         public MainForm()
41         {
42             // WinForm Init
43             InitializeComponent();
44
45             // Initialize the image resources
46             InitializeImages();
47
48             // Initialize the settings object
49             _mainFormSettings = new SettingsHelper();
50
51             // Call the reset button click event handler to open the settings window before main form load.
52             BtnRestartClickHandler(this, EventArgs.Empty);
53         }
54     }
55 }

```

Figure 18: Screenshot of the citations and constructor for the main form.

```

56  /// <summary>
57  /// Method for initializing all the panel images
58  /// </summary>
59  /// </summary>
60  private void InitializeImages()
61  {
62      // Pre-load all the image resources for the cells
63      _blankCellImage = Properties.Resources.Minesweeper_Blank_Cell;
64      _oneNeighborImage = Properties.Resources.Minesweeper_1;
65      _twoNeighborImage = Properties.Resources.Minesweeper_2;
66      _threeNeighborImage = Properties.Resources.Minesweeper_3;
67      _fourNeighborImage = Properties.Resources.Minesweeper_4;
68      _fiveNeighborImage = Properties.Resources.Minesweeper_5;
69      _sixNeighborImage = Properties.Resources.Minesweeper_6;
70      _sevenNeighborImage = Properties.Resources.Minesweeper_7;
71      _eightNeighborImage = Properties.Resources.Minesweeper_8;
72      _nineNeighborImage = Properties.Resources.Minesweeper_9;
73      _bombImage = Properties.Resources.Minesweeper_Bomb_Cell;
74      _flaggedCellImage = Properties.Resources.Minesweeper_Flagged_Cell;
75      _hiddenCellImage = Properties.Resources.Minesweeper_Hidden_Cell;
76  }
77
78  /// <summary>
79  /// restart button click event handler
80  /// </summary>
81  /// </summary>
82  /// <param name="sender"></param>
83  /// <param name="e"></param>
84  /// </summary>
85  private void BtnRestartClick(object sender, EventArgs e)
86  {
87      Form setup = new UI.Forms.SetupForm(mainFormSettings);
88      setup.ShowDialog();
89      FillPanelWithCells();
90  }

```

Figure 19: Screenshot of the Initialize images and restart button click event handler.

```

87  /// <summary>
88  /// Method to fill the parent panel pnlMain with all the child panels that will represent the cells
89  /// </summary>
90  /// </summary>
91  private void FillPanelWithCells()
92  {
93      // Board size = size setting (4, 8, 12, 16, etc...)
94      // Number of cell = board size, each cell is 25, 25 in size
95      // Board panel starts at loc 12, 12 and is size 500. Form starts at size 600, 500
96
97      // Declare and initialize
98      pnlMain.Controls.Clear();
99      _board = new BoardLogic(_mainFormSettings.BoardSize);
100
101      // Setup the board
102      _board.SetDifficulty(_mainFormSettings.Difficulty);
103      _board.SetupBombs();
104      _board.CountBombsNearby();
105
106      // Initialize the values of the labels holding bomb and reward data
107      lblBombValue.Text = _board.GetNumberofBombs().ToString("00");
108      lblRewardValue.Text = _board.GetNumberofRewards().ToString("00");
109
110      // Adjust the size of the form depending on the size of the board. If size is
111      // catch super small boards and make them big enough to fit all the other controls
112      if (_mainFormSettings.BoardSize > 4)
113      {
114          this.Size = new Size((_mainFormSettings.BoardSize * 25) + 180, (_mainFormSettings.BoardSize * 25) + 60);
115      }
116      else // Small board so we will use the minimum size
117      {
118          this.Size = new Size((_mainFormSettings.BoardSize * 25) + 180, (_mainFormSettings.BoardSize * 25) + 160);
119      }
120
121      // Pause pnlMain layout until we add all the children
122      pnlMain.SuspendLayout();
123      // Iterate over all the cells and add the appropriate panel
124      for (int row = 0; row < _board.GetBoardSize(); row++)
125      {
126          for (int col = 0; col < _board.GetBoardSize(); col++)
127          {
128              Panel panel = new Panel();
129              panel.Size = new Size(25, 25);
130              panel.Location = new Point((row * 25), (col * 25));
131              panel.BackgroundImage = _hiddenCellImage;
132              panel.BackgroundImageLayout = ImageLayout.Stretch;
133              panel.Tag = new Point(row, col);
134              panel.Click += PanelClick;
135              pnlMain.Controls.Add(panel);
136          }
137      }
138      // Resume the layout of the pnlMain
139      pnlMain.ResumeLayout();
140  }

```

Figure 20: Screenshot of the fill panel with cells method. This method is called once per game and sets the main forms size, fills the main panel with all the child panels, and does all the other heavy lifting required to start a new game.

```

142 /// <summary>
143 /// Handle the click event for the panel cells
144 /// </summary>
145 /// <param name="sender"></param>
146 /// <param name="e"></param>
147 <summary>
148 private void PanelCellClickEventHandler(object? sender, EventArgs e)
149 {
150     // Capture which button is clicked
151     MouseEventArgs args = (MouseEventArgs)e;
152     MouseButton userClicked = args.Button;
153
154     // Pull the cells location out of the panels tag
155     Panel panel = (Panel)sender;
156     Point cellLoc = (Point)panel.Tag;
157
158     // If the game is still in progress use mouse button to determine the proper next action.
159     if (_board.GetGameState() == GameState.InProgress)
160     {
161         if (userClicked == MouseButton.Left) // User Left clicked
162         {
163             // Left click signifies visit. Call DetermineGameState with a visit command of 1
164             _board.DetermineGameState(cellLoc.X, cellLoc.Y, 1);
165             // Refresh pnlMain layout
166             RefreshPanels(panel);
167             // Remove any border styles applied previously by using rewards
168             panel.BorderStyle = BorderStyle.None;
169         }
170         else if (userClicked == MouseButton.Right) // User Right clicked
171         {
172             // Right click signifies flag. Call DetermineGameState with a flag command of 2
173             _board.DetermineGameState(cellLoc.X, cellLoc.Y, 2);
174             // Refresh pnlMain layout
175             RefreshPanels(panel);
176             // Remove any border styles applied previously by using rewards
177             panel.BorderStyle = BorderStyle.None;
178         }
179         else // User clicked any other mouse button (use reward)
180         {
181             // Make sure there are rewards left
182             if (_board.GetNumberOfRewards() > 0)
183             {
184                 // Other clicks signify use reward. Call DetermineGameState with a reward command of 3
185                 _board.DetermineGameState(cellLoc.X, cellLoc.Y, 3);
186                 // Refresh pnlMain layout
187                 RefreshPanels(panel);
188                 // Set the panel's border property to fixed single so we can remember what cells had reward used on
189                 panel.BorderStyle = BorderStyle.FixedSingle;
190             }
191             else
192             {
193                 // Catch no rewards left
194                 MessageBox.Show("You are out of rewards. ");
195                 return;
196             }
197         }
198     }
199     // Check state for win loss
200     StateCheck();
201     // cell visit again to clear reward game states
202     if (_board.GetGameState() == GameState.RewardFound)
203     {
204         _board.DetermineGameState(cellLoc.X, cellLoc.Y, 1);
205     }
206 }

```

Figure 21: Screenshot of the panel cell click event handler. This event handler uses the button information passed in from EventArgs to determine the action to perform based on which button the user clicked. It calls the boards determine state method, refresh panels, and state check before finishing.

```

287 /// <summary>
288 /// Method to catch the different game states and notify the user of what happened.
289 /// </summary>
290 <summary>
291 private void StateCheck()
292 {
293     switch (_board.GetGameState())
294     {
295         case GameState.Lost:
296             // Reveal all the bombs on the board for the user after a loss
297             RevealAllBombs();
298             MessageBox.Show("You lost try again.");
299             break;
300         case GameState.Won:
301             MessageBox.Show("You Won!! Great Job.");
302             break;
303         case GameState.RewardFound:
304             MessageBox.Show("You found a reward. ");
305             break;
306         default:
307             break;
308     }
309 }
310
311 /// <summary>
312 /// Show all bomb locations to the user after a loss
313 /// </summary>
314 <summary>
315 private void RevealAllBombs()
316 {
317     foreach (Panel panel in pnlMain.Controls)
318     {
319         // Get the board cell from the panel tag
320         Point cellLoc = (Point)panel.Tag;
321         CellModel cell = _board.GetCellAt(cellLoc.X, cellLoc.Y);
322
323         if (cell.IsBomb)
324         {
325             panel.BackgroundImage = _nineNeighborsIsBombImage;
326         }
327     }
328 }

```

Figure 22: Screenshot of the state check and reveal all bombs methods. State check catches win, loss, and reward events to display the result to the user. Reveal all bombs is called if the user loses to show the location of all the bombs.


```

251 > <summary> Refreshes the image state of all the panels on the board
252 </summary>
253 <param name="primaryPanel" type="Panel" />
254 private void RefreshPanels(Panel primaryPanel)
255 {
256     // Declare and Initialize
257     Point primaryLoc = (Point)primaryPanel.Tag;
258     CellModel primaryCell = _board.GetCellAt(primaryLoc.X, primaryLoc.Y);
259     // Short cut check for flood fill. If we have neighbors simply paint the panel.
260     // Else we found a void and will iterate over all the panels to update them all.
261     if (!primaryCell.NumberOfBombNeighbors > 0)
262     {
263         PaintPanel(primaryPanel, primaryCell);
264     }
265     else
266     {
267         foreach (Panel panel in pnlMain.Controls)
268         {
269             // Get the board cell from the panel tag
270             Point cellLoc = (Point)panel.Tag;
271             CellModel cell = _board.GetCellAt(cellLoc.X, cellLoc.Y);
272             PaintPanel(panel, cell);
273         }
274     }
275 }
276
277 // Update the bomb and reward tracking labels
278 lblBombsValue.Text = _board.GetNumberOfBombs().ToString("00");
279 lblRewardsValue.Text = _board.GetNumberOfRewards().ToString("00");
280
281 <summary>
282 <param name="panel" type="Panel" />
283 <param name="cell" type="CellModel" />
284 private void PaintPanel(Panel panel, CellModel cell)
285 {
286     // If it is a flag flag it else
287     if (cell.IsFlagged)
288     {
289         panel.BackgroundImage = _flaggedCellImage;
290     }
291     // If it is visited, alter the image based on the case
292     else if (cell.IsVisited)
293     {
294         switch (cell.NumberOfBombNeighbors)
295         {
296             case 0:
297                 panel.BackgroundImage = _emptyCellImage;
298                 break;
299             case 1:
300                 panel.BackgroundImage = _oneBombImage;
301                 break;
302             case 2:
303                 panel.BackgroundImage = _twoBombsImage;
304                 break;
305             case 3:
306                 panel.BackgroundImage = _threeBombsImage;
307                 break;
308             case 4:
309                 panel.BackgroundImage = _fourBombsImage;
310                 break;
311             case 5:
312                 panel.BackgroundImage = _fiveBombsImage;
313                 break;
314             case 6:
315                 panel.BackgroundImage = _sixBombsImage;
316                 break;
317             case 7:
318                 panel.BackgroundImage = _sevenBombsImage;
319                 break;
320             case 8:
321                 panel.BackgroundImage = _eightBombsImage;
322                 break;
323             default:
324                 break;
325         }
326     }
327     // If a cell is not flagged and it is not visited it is hidden.
328     // this became necessary because removing a flag was breaking things
329     // I was trying to avoid painting hidden cells every move but I ended up needing to
330     else
331     {
332         panel.BackgroundImage = _hiddenCellImage;
333     }
334 }

```

Figure 23: Screenshot of the Refresh panel and paint panel methods. Paint is called by refresh. The two methods were originally one method but I separated them to cut down on the size of refresh and make it more readable.

```

17 public class BoardLogic
18 {
19     // private class level variables
20     private BoardModel _board;
21
22     /// <summary> Public Constructor that takes one param of size
23     public BoardLogic(int size)
24     {
25         // <summary> The method for setting up the bombs and rewards on the board
26         SetupBombs();
27
28         // <summary> Method to count neighbor bombs
29         CountBombsNearby();
30
31         // <summary> Public accessor for retrieving cells at locations in the grid
32         GetCellAt(int row, int col);
33
34         // <summary> Public accessor for acquiring the boards size
35         GetBoardSize();
36
37         // <summary> Sets the games difficulty
38         SetDifficulty(int difficulty);
39
40         // <summary> Determines the results for the location and action the user picked
41         DetermineGameState(int row, int col, int choice);
42
43         // <summary> Method to handle cell visit cases
44         VisitCell(CellModel cell);
45
46         // <summary> Method to handle cell flagging cases
47         FlagCell(CellModel cell);
48
49         // <summary> Provides functionality for special reward usage
50         UseSpecialBonus(CellModel cell);
51
52         // <summary> Not yet implemented
53         DetermineFinalScore();
54
55         // <summary> Counts the remaining moves when value hits zero player has won
56         CountRemainingMoves();
57
58         // <summary> provides access to the number of bombs in the board model
59         GetNumberOfBombs();
60
61         // <summary> Provides access to the number of collected rewards that are stored
62         GetNumberOfRewards();
63
64         // <summary> Accessor for the internal game state of the board
65         GetGameState();
66
67         // <summary> Flood fill algorithm for filling unoccupied squares
68         FloodFill(int row, int col);
69
70         // <summary> Method for setting all the flags when a reward is found
71         ClaimReward(int row, int col);
72     }
73 }

```

Figure 24: Screenshot of the board logic class. It is largely unchanged from previous versions. Changes are shown below in Figure 25.

```

261 /// <summary>
262 /// Method to handle cell flagging cases
263 /// </summary>
264 /// <param name="cell">CellModel</param>
265 public void FlagCell(CellModel cell)
266 {
267     // Are there bombs still on the board or are we removing a possibly incorrect flag
268     if (_board.NumberOfBombs > 0 && !cell.IsVisited || cell.IsFlagged)
269     {
270         // Flip flop the flag state
271         cell.IsFlagged = !cell.IsFlagged;
272
273         // Update bomb count and is visited property based on our displayed flags
274         if (cell.IsFlagged && !cell.IsVisited)
275         {
276             _board.NumberOfBombs--;
277             cell.IsVisited = true;
278         }
279         else
280         {
281             _board.NumberOfBombs++;
282             cell.IsVisited = false;
283         }
284     }
285     else if (cell.IsVisited)
286     {
287         MessageBox.Show("You can't flag a revealed cell. ");
288     }
289     else
290     {
291         MessageBox.Show("You are out of flags. ");
292     }
293 }
294
295 /// <summary>
296 /// Provides functionality for special reward usage
297 /// </summary>
298 /// <param name="cell">CellModel</param>
299 public void UseSpecialBonus(CellModel cell)
300 {
301     Console.ForegroundColor = ConsoleColor.DarkBlue;
302     if (cell.IsBomb)
303     {
304         MessageBox.Show("That cell does have a bomb");
305     }
306     else
307     {
308         MessageBox.Show("That cell does not have a bomb");
309     }
310     Console.ResetColor();
311     _board.RewardsRemaining--;
312 }

```

Figure 25: Screenshot of the flag cell and use special bonus methods from the board logic. Both of these classes previously used console write line to output information to the user. I now have them using message box show. I don't like using ui elements like message box here in the business layer but I unsure of a proper way to implement it. I considered more game state flags

in that enumeration but I don't like that idea. I have run into things like this in the past and I have always struggled with figuring out a proper and elegant way to solve it. I am open to ideas but not sure on the best way.

The key concepts demonstrated in this week's milestone are multi-form navigation, event-driven programming, and separation of concerns. This week we transitioned from a console application to an event driven windows forms application with a GUI. We demonstrated how an application written with proper separation of concerns can use the same back-end element (BoardLogic) to drive completely different looking front ends. We demonstrated how to use multiple forms to pass information in one form to another. The milestone showcased event-driven programming through button click handlers and dynamic UI updates, where the game board state is continuously reflected in the visual interface via a programmed refresh method.